

General Disclosure

The GAI tool used was Google Gemini 3.1 Pro.

This tool was consulted on 9/25/2026.

Code Attribution

Below is the exact prompt listed to generate the code. For the sake of authenticity, I have provided the prompt and its resulting output in the form of screenshots.

Write a C++ program that reads a line of text and extracts a single valid IPv4 address — optionally followed by a port number — embedded anywhere in that text. Only digits, periods (.), and colons (:) are ever part of a valid token; every other character is garbage and is skipped. A candidate token must match the address grammar in full — no partial matches, no truncating to find a valid piece inside a longer run. This program must be done in a continuous loop where the user is prompted for input until the user enters END (case-sensitive), then prints "Program Terminated." and exits. Main() function should receive the result from extractIPv4() and format the output exactly as directed below.

The function used to extract the ip address must implemented exactly as described below, with all digit accumulation being done by hand.

The function returns whether a valid address was found, and delivers the address and port through two additional parameters:

```
// Returns True if a valid address was found, False otherwise.  
// On success: outAddress holds the 32-bit value,  
// and outPort holds the port number, or -1 if no port was  
// present.  
// On failure: outAddress is set to 0 and outPort is set to -1.  
bool extractIPv4(const std::string& str, unsigned long&  
outAddress, int& outPort);
```

In terms of what describes an address: An address is four octets separated by periods (octet.octet.octet.octet), each octet 1–3 digits, value 0–255, no leading zero unless the value is exactly 0. An optional :port may follow the fourth octet: 1–5 digits, value 0–65535, same leading-zero rule. If a colon is present, the port must be fully valid or the entire match — address included — is rejected.

Additional requirements need to be met for this program, the

Additional requirements need to be met for this program, the first is a limitation of functions and libraries that may be used: Any string-to-number conversion function: `atoi`, `atol`, `atoll`, `strtol`, `strtoul`, `strtod`, `stoi`, `stol`, `stoul`, `sscanf`, `scanf` with numeric conversions. Any address-parsing library function: `inet_aton`, `inet_pton`, `inet_addr`, or equivalents. Any regular-expression facility (`std::regex`, `POSIX regex.h`, or similar) — the parsing and validation logic must be your own character-by-character code, not a pattern matched by a library. Standard character-classification functions (`isdigit`, etc.) are fine to use.

Other requirements are as follows:

Exactly one valid address may be extracted per input line; everything else in the line is either garbage (skipped) or part of a candidate token that fails validation.

Reject anything that does not exactly match the grammar above — wrong octet count, empty octet, out-of-range octet or port, a disallowed leading zero, a second colon, a colon not immediately after the fourth octet, or a stray period/colon directly adjacent to an otherwise-valid address.

On success where `extractIPv4()` returns `True`, print: 'Extracted IPv4 address: A.B.C.D (decimal value: N, port: P)' where N is the 32-bit decimal value and P is the port number or the literal text `none`. A, B, C, and D are the octets respectively.

An example run consisting of the user's input and corresponding output can be found here:

```
Enter a string (or 'END' to quit): connecting to 192.168.1.1 now
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777,
port: none)
Enter a string (or 'END' to quit): server=10.0.0.255:8080end
Extracted IPv4 address: 10.0.0.255 (decimal value: 167772415,
port: 8080)
Enter a string (or 'END' to quit): 192a168.1.1
Extracted IPv4 address: 168.1.1.1 (decimal value: 2818638081,
port: none)
```

not immediately after the fourth octet, or a stray period/colon directly adjacent to an otherwise-valid address. On success where `extractIPv4()` returns `True`, print: 'Extracted IPv4 address: A.B.C.D (decimal value: N, port: P)' where N is the 32-bit decimal value and P is the port number or the literal text `none`. A, B, C, and D are the octets respectively.

An example run consisting of the user's input and corresponding output can be found here:

```
Enter a string (or 'END' to quit): connecting to 192.168.1.1 now
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777,
port: none)
Enter a string (or 'END' to quit): server=10.0.0.255:8080end
Extracted IPv4 address: 10.0.0.255 (decimal value: 167772415,
port: 8080)
Enter a string (or 'END' to quit): 192a168.1.1
Extracted IPv4 address: 168.1.1 (decimal value: 2818638081,
port: none)
Enter a string (or 'END' to quit): 192.168.1.1.
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): Connection from 192.168.1.1
refused
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777,
port: none)
Enter a string (or 'END' to quit): 192.168.01.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:99999
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 12.34.56
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): no number here
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): END
Program terminated.
```



Here is the complete C++ program that fulfills all your requirements. It implements the custom, character-by-character tokenization and validation logic without using any of the banned standard conversion or regex libraries.

C++



```
#include <iostream>
#include <string>

// Helper function to validate a single token and parse its values if valid.
// Returns true if the token is a strictly valid IPv4 address (with optional port).
bool isValidToken(const std::string& t, unsigned long& addr, int& port) {
    if (t.empty()) return false;

    int parts[5];
    int numParts = 0;

    unsigned long currentVal = 0;
    int digitCount = 0;
    bool isPortPhase = false;

    for (size_t i = 0; i < t.length(); ++i) {
        char c = t[i];

        if (c == '.') {
            if (isPortPhase) return false; // Dot not allowed in port
            if (digitCount == 0) return false; // Consecutive dots or leading dot

            // Leading zero check: if more than 1 digit, first digit cannot be '0'
            if (digitCount > 1 && t[i - digitCount] == '0') return false;
            if (currentVal > 255) return false;

            parts[numParts++] = static_cast<int>(currentVal);
            if (numParts > 4) return false; // Too many octets

            currentVal = 0;
            digitCount = 0;
        }
        else if (c == ':') {
            if (isPortPhase) return false; // Multiple colons
            if (digitCount == 0) return false; // Colon immediately after a dot or s
```

+ Ask Gemini

Pro



Gemini is AI and can make mistakes.

C++



```
    else if (c == ':') {
        if (isPortPhase) return false; // Multiple colons
        if (digitCount == 0) return false; // Colon immediately after a dot or s

        if (digitCount > 1 && t[i - digitCount] == '0') return false;
        if (currentVal > 255) return false;

        parts[numParts++] = static_cast<int>(currentVal);
        if (numParts != 4) return false; // Colon must be exactly after the 4th

        isPortPhase = true;
        currentVal = 0;
        digitCount = 0;
    }
    else if (c >= '0' && c <= '9') {
        digitCount++;
        currentVal = currentVal * 10 + (c - '0');

        if (!isPortPhase) {
            // Octet checks
            if (digitCount > 3 || currentVal > 255) return false;
        } else {
            // Port checks
            if (digitCount > 5 || currentVal > 65535) return false;
        }
    }
    else {
        // Should never be reached based on tokenization rules, but included for
        return false;
    }
}

// Process the final segment of the token
if (digitCount == 0) return false; // Token ends with a '.' or ':'
if (digitCount > 1 && t[t.length() - digitCount] == '0') return false;

if (!isPortPhase) {
    if (currentVal > 255) return false;
    parts[numParts++] = static_cast<int>(currentVal);
    if (numParts != 4) return false;
    port = -1;
} else {
    if (currentVal > 65535) return false;
    port = static_cast<int>(currentVal);
}
```

+ Ask Gemini

Pro



Gemini is AI and can make mistakes.

C++



```
        if (numParts != 4) return false;
        port = -1;
    } else {
        if (currentVal > 65535) return false;
        parts[numParts++] = static_cast<int>(currentVal);
        if (numParts != 5) return false;
        port = parts[4];
    }

    // Assemble the 32-bit address
    addr = ((unsigned long)parts[0] << 24) |
           ((unsigned long)parts[1] << 16) |
           ((unsigned long)parts[2] << 8) |
           ((unsigned long)parts[3]);

    return true;
}

// Returns True if a valid address was found, False otherwise.
// On success: outAddress holds the 32-bit value,
// and outPort holds the port number, or -1 if no port was present.
// On failure: outAddress is set to 0 and outPort is set to -1.
bool extractIPv4(const std::string& str, unsigned long& outAddress, int& outPort) {
    outAddress = 0;
    outPort = -1;

    std::string currentToken = "";

    // We iterate up to str.length() inclusively to force processing of the last token
    for (size_t i = 0; i <= str.length(); ++i) {
        char c = (i < str.length()) ? str[i] : ' ';

        // Allowed characters for a candidate token
        if ((c >= '0' && c <= '9') || c == '.' || c == ':') {
            currentToken += c;
        } else {
            // End of current token sequence
            if (!currentToken.empty()) {
                unsigned long tempAddr = 0;
                int tempPort = -1;

                if (isValidToken(currentToken, tempAddr, tempPort)) {
                    outAddress = tempAddr;
                }
            }
            currentToken = "";
        }
    }
}
```

+ Ask Gemini

Pro



Gemini is AI and can make mistakes

C++



```
        if (isValidToken(currentToken, tempAddr, tempPort)) {
            outAddress = tempAddr;
            outPort = tempPort;
            return true;
        }
        // Reset token for the next candidate search
        currentToken = "";
    }
}

return false;
}

int main() {
    std::string inputLine;

    while (true) {
        std::cout << "Enter a string (or 'END' to quit): ";
        if (!std::getline(std::cin, inputLine)) {
            break; // Break gracefully on unexpected EOF
        }

        if (inputLine == "END") {
            std::cout << "Program terminated.\n";
            break;
        }

        unsigned long address = 0;
        int port = -1;

        if (extractIPv4(inputLine, address, port)) {
            // Unpack 32-bit int back into individual octets for display purposes
            unsigned long a = (address >> 24) & 0xFF;
            unsigned long b = (address >> 16) & 0xFF;
            unsigned long c = (address >> 8) & 0xFF;
            unsigned long d = address & 0xFF;

            std::cout << "Extracted IPv4 address: "
                << a << "." << b << "." << c << "." << d
                << " (decimal value: " << address << ", port: ";

            if (port == -1) {
```

+ Ask Gemini

Pro



```
C++  
  
std::string inputLine;  
  
while (true) {  
    std::cout << "Enter a string (or 'END' to quit): ";  
    if (!std::getline(std::cin, inputLine)) {  
        break; // Break gracefully on unexpected EOF  
    }  
  
    if (inputLine == "END") {  
        std::cout << "Program terminated.\n";  
        break;  
    }  
  
    unsigned long address = 0;  
    int port = -1;  
  
    if (extractIPv4(inputLine, address, port)) {  
        // Unpack 32-bit int back into individual octets for display purposes  
        unsigned long a = (address >> 24) & 0xFF;  
        unsigned long b = (address >> 16) & 0xFF;  
        unsigned long c = (address >> 8) & 0xFF;  
        unsigned long d = address & 0xFF;  
  
        std::cout << "Extracted IPv4 address: "  
            << a << "." << b << "." << c << "." << d  
            << " (decimal value: " << address << ", port: ";  
  
        if (port == -1) {  
            std::cout << "none)\n";  
        } else {  
            std::cout << port << ")\n";  
        }  
    } else {  
        std::cout << "Invalid input: no valid IPv4 address found\n";  
    }  
}  
  
return 0;  
}
```

🏠 🔍 ↻ 📄 ...

+ Ask Gemini Pro 🔊

Gemini is AI and can make mistakes.

No modifications were made to the code beyond the inclusion of additional comments to explain functionality.

A modification was initially considered: the refactoring of isValidToken() into extractIPv4() as one singular function. This modification was not made as the separation of isValidToken()'s functionality for verifying if a perceived address is in the correct format did not violate the given function prototype. The function, extractIPv4(), still returns whether a valid address was found and the address and port through two additional parameters.

Verification Statement

I can confirm that I, Kristoffer Roy Comahig, understand every line of submitted code. This understanding is demonstrated with the added comments with the prefix 'STU:' that denote areas of code that were not commented and explained during the initial generation of the code. This code has been tested with the test case inputs included within the submitted repository and is working as intended.

At the time of writing this statement, there are no known bugs, limitations, or unexpected behaviors that were not resolved.